

Meridian

Product Requirements Document (Week 1 Demo) — v2

An AI-native workspace where every answer is grounded, cited and inspectable

Author	Esandu Obadaarachchi
Programme	DigitalT3 Incubation Pod — Week 1 Kavia Build
Status	Draft v2 — revised after pod review, for sign-off
Last updated	15 September 2026

RELIABILITY NOTE — must appear in UI, README and demo

AI-generated answers and agent actions may be incomplete or wrong. Every answer carries a chunk-level source citation and a stated, uncalibrated confidence value. Low-confidence or ungrounded answers are flagged for human review rather than shown as fact.

Revision note: *this version replaces the non-goals list that quietly cancelled itself with a real MUST/SHOULD/COULD scope, adds two build gates with named cut rules, an evaluation section with a golden set and an injection red-team suite, and fixes the data model gaps the review identified (chunk-level citations, retrieval run logging, undo, role modelling, the block editor's storage format).*

Table of Contents

1.	One-line summary
2.	Problem
3.	Scope: MUST / SHOULD / COULD
4.	Build gates & cut rules
5.	Evaluation
6.	Target users & roles
7.	Key user journeys (MUST scope)
8.	Functional requirements
9.	AI approach
10.	Technical architecture
11.	Data model (Supabase / Postgres)
12.	Core API endpoints (FastAPI)
13.	Guardrails (must-have)
14.	Risks & latency
15.	Why this is different
16.	Five-day build plan
17.	Open decisions

1. One-line summary

Meridian is an AI-native workspace: a block-based note editor and task list, paired with an agent that answers questions with chunk-level citations and proposes actions held for human approval before anything changes. Low-confidence or ungrounded answers are flagged for review, never presented as fact.

2. Problem

Knowledge work is scattered across separate tools: notes in one app, tasks in another, documents in a shared drive. The current generation of AI layers bolted onto these tools, Notion Agent, ClickUp's Brain², Tana and others, now complete real multi-step actions on a user's behalf.

Several of them show a source link. Few show the depth behind an action: how a candidate was retrieved, why it was trusted, what confidence the system actually had, or proof that an uploaded document could not hijack the agent's next step. An agent that can create tasks or answer questions for a team should be able to show its work at that depth, not just deliver a confident-sounding result.

Meridian is designed to close that specific gap: an open, inspectable retrieval and reasoning pipeline behind every answer and every action the agent proposes.

3. Scope: MUST / SHOULD / COULD

Replaces the old non-goals list. Nothing below is aspirational: MUST is the spine the Friday demo has to prove. SHOULD is built only if a gate in section 4 passes early. COULD is not built in Week 1 at all.

MUST — the Week 1 spine

- Google sign-in, workspace-scoped auth
- Block-based note editor, notes stored as a real block tree
- Document upload (PDF, Word), chunked into a real chunks table
- Hybrid retrieval: dense plus sparse search, rerank, grading, retry, groundedness self-check
- Ask-the-agent with clickable, chunk-level citations and a defined, stated confidence value
- Agent proposes a task from a note or conversation, held for explicit approval, written with full reasoning once approved
- Full audit log, admin review queue for flagged answers and actions
- Prompt-injection defence built as structural isolation, proven with a red-team suite
- Pipeline health dashboard backed by a real `retrieval_runs` table

SHOULD — built only if a gate passes early

- Backlog and sprint board: tasks move between a backlog and a dated sprint, ticked through To Do, In Progress, Done
- Automatic task assignment: agent reads `workspace_members`' team role and proposes an assignee, reasoning shown

- Undo on agent-written changes, using a stored before-state

COULD — not built in Week 1

- Whiteboard canvas. Approach: Excalidraw-style, own timeline once the agent core is proven
- Flow / diagram view. Approach: React Flow-based, same
- Google Calendar sync. Needs a sync mechanism decided (poll vs webhook), token storage and OAuth scopes defined before it is worth a day of work

4. Build gates & cut rules

GATE G1 — checked end of Day 3

Pass: retrieval answers at least 12 of the 15 golden questions (section 5) with the correct source chunk cited.

If it fails: cut the retry loop, ship rerank-only for the demo, state this plainly in the write-up and the demo narration.

GATE G2 — checked end of Day 4

Pass: the agent propose-then-approve round trip works end to end for at least one real task.

If it fails: switch to auto-execute with full reasoning logged, cut the approval UI for Week 1, carry it forward as a SHOULD.

5. Evaluation

- **Golden set:** 15 hand-written questions against the uploaded documents, each with the correct source chunk labelled by hand before any model runs. Retrieval and groundedness are scored against this set, not against impressions.
- **Injection red-team suite:** 8 documents, each with a prompt injection attempt embedded (for example, an instruction telling the agent to ignore its rules and take an action). Run through the full pipeline before Day 5. Target: 8 of 8 caught, reported honestly either way.
- **Confidence value:** combined reranker score and grading verdict, shown in the UI. Stated plainly in the UI and the README as uncalibrated in Week 1, not a probability.
- **Latency budget:** target p50 under 8 seconds per question, end to end. First cut if this is missed is the retry loop, the same cut Gate G1 already names.

6. Target users & roles

Personas

- **Member:** creates notes and tasks, asks the agent questions, reviews cited answers.
- **Admin:** reviews flagged or low-confidence answers and actions, checks the audit log, manages workspace members.

Roles (Auth)

Member and Admin are scoped per workspace on `workspace_members`, not stored globally on the user. A person can hold different roles in different workspaces.

7. Key user journeys (MUST scope)

7.1 Ask and get a chunk-cited answer

1. Member signs in with Google.
2. Uploads or already has documents and notes in the workspace.
3. Asks the agent a question.
4. Agent rewrites the query, retrieves candidates via hybrid search, reranks, grades relevance, retries if the first pass is weak, runs a groundedness check before responding.
5. Answer is shown with clickable citations linking to the exact source chunk, plus a stated confidence value; low-confidence answers are flagged rather than stated as fact.

7.2 Agent proposes a task, held for approval

1. Member asks the agent to create a task from a note or conversation.
2. Agent proposes the task with its reasoning shown; nothing is written yet.
3. Member or Admin approves or rejects the proposal.
4. On approval, the task is written to the workspace and logged to the audit trail. On rejection, nothing is written, and the rejection itself is logged.

7.3 Admin review

1. Admin opens the review panel.
2. Sees a queue of low-confidence or flagged answers and pending agent actions.
3. Confirms, corrects or dismisses each one.
4. Corrections are available to inform future retrieval tuning.

8. Functional requirements

8.1 Member-facing (MUST)

- Google sign-in
- Block-based note editor, notes stored as a block tree
- Upload documents (PDF, Word)
- Ask-the-agent panel with clickable chunk-level citations and a stated confidence value
- Task list showing agent-proposed tasks pending approval, and approved tasks

8.2 Admin-facing (MUST)

- Review queue for low-confidence or flagged answers and pending agent actions

- Audit log: predictions, agent actions and admin decisions, timestamped and attributable
- Workspace member management: invite, set Member or Admin role
- Pipeline health view, backed by real `retrieval_runs` data: retrieval hit rate and groundedness pass rate over time

8.3 System requirements (MUST)

- Retrieval pipeline runs on every question asked
- Every answer stored with its chunk citations, confidence and groundedness-check result
- Guardrail routing: below threshold, always flagged for review rather than shown as a confident answer
- Every agent-proposed write action is held for approval; nothing is written to the workspace before a Member or Admin approves it
- Structural injection isolation active on every document ingested, not optional per document

SHOULD-tier requirements (team roster with job-function role, backlog/sprint board, undo) are listed in section 3 and built only if Gate G1 or G2 pass with room to spare.

9. AI approach

- **Orchestration:** agent loop (LangGraph) with a small tool set: search the workspace, read a task, propose a task write (held for approval)
- **Models:** routed through a thin provider gateway, swappable, with a response cache so repeated demo runs are fast and cheap. A fast, low-cost model drives the agent's step-by-step loop; a stronger model handles final answer generation and citation assembly
- **Retrieval:** dense embeddings plus a sparse keyword signal combined (hybrid search), a reranking step narrows candidates, a grading step checks whether retrieved context actually supports the question, a retry loop re-queries if not, a groundedness self-check runs before the answer is shown
- **Citations:** every chunk that contributes to an answer is recorded in `answer_citations`, linking the answer to the exact chunk and its character offsets in the source document, so a citation click opens the precise passage, not just the file
- **Confidence:** combined reranker score and grading verdict, shown as an explicit, stated-uncalibrated value, never a bare unexplained percentage
- **Guardrails:** document and tool-returned content is passed to the model as data, never as instructions; the agent's tool-call decisions are only ever triggered by the user's own turns; the injection red-team suite (section 5) is run against this design before Day 5
- **Parsing:** PDF and Word for Week 1

10. Technical architecture

10.1 Stack

- **Frontend:** React, deployed to Vercel
- **Backend:** FastAPI (retrieval and agent orchestration), deployed to Railway

- **DB + Storage:** Supabase (Postgres + Supabase Storage), row-level security enforced on every table by `workspace_id`
- **Auth:** Supabase Auth with Google OAuth
- **Vector search:** managed vector DB with hybrid dense and sparse support
- **Orchestration:** LangGraph, behind a provider gateway

The schema supports multiple workspaces per account structurally, since RLS is scoped per workspace regardless. The Week 1 demo exercises a single active workspace per account; multi-workspace switching is not built.

10.2 High-level flow

1. Document uploaded → stored in Supabase, chunked, embedded, written to chunks.
2. Member asks a question → agent loop starts, run recorded in `retrieval_runs`.
3. Hybrid retrieval, rerank, grade, retry if needed, groundedness check → answer returned with citations.
4. Agent action proposed → held, not written, until approved; approval or rejection logged to `audit_log`.
5. Admin review → decision written to `admin_reviews`.

11. Data model (Supabase / Postgres)

Table	Fields
users	id, email, created_at
workspaces	id, name, owner_id, created_at
workspace_members	id, workspace_id, user_id, auth_role, team_role, joined_at
documents	id, workspace_id, uploaded_by, file_path, parsed_status, created_at
chunks	id, document_id, content, char_start, char_end, embedding_ref, created_at
notes	id, workspace_id, created_by, content (jsonb block tree), updated_at
tasks	id, workspace_id, sprint_id, title, status, assignee_id, due_date, created_by, source, created_at
sprints	id, workspace_id, name, start_date, end_date, status, created_at
agent_answers	id, workspace_id, question, answer, confidence, groundedness_pass, created_at
answer_citations	id, answer_id, chunk_id, created_at
agent_actions	id, workspace_id, action_type, target_table, target_id, reasoning, before_state, status, created_at

Table	Fields
retrieval_runs	id, workspace_id, question, candidates_json, rerank_scores_json, grade_outcome, retry_count, latency_ms, created_at
audit_log	id, actor_id, actor_type, action, target_id, timestamp, details
admin_reviews	id, review_target_type, review_target_id, reviewer_id, decision, notes, reviewed_at

- `users.role` removed; auth role now lives on `workspace_members.auth_role`, since role is inherently workspace-scoped
- `notes.block_type` removed; a block-based editor needs a block tree, not one type per row, so content is now the full jsonb block structure
- `agent_answers.sources` replaced by `answer_citations`, a real join to chunks rather than a JSON blob
- `agent_actions.before_state` added so undo (SHOULD) has something to revert to, if it is built
- `audit_log.actor_type` added (user / agent / system) since the agent itself needs to appear as an actor, not only human users

12. Core API endpoints (FastAPI)

Member-facing (MUST)

Method	Path	Description
POST	/documents/upload	Upload and parse a document
POST	/notes	Create or update a note
POST	/agent/ask	Ask the agent a grounded question
GET	/tasks	List tasks, including pending approvals
POST	/agent/actions/{id}/approve	Approve a held action; writes it and logs it
POST	/agent/actions/{id}/reject	Reject a held action; logs it, writes nothing

Admin-facing (MUST)

Method	Path	Description
GET	/admin/review-queue	Flagged answers and pending actions
POST	/admin/reviews/{target_type}/{target_id}	Submit a review decision
GET	/admin/audit-log	Full audit trail
GET	/admin/pipeline-health	Metrics from retrieval_runs

Method	Path	Description
POST	/admin/members	Invite a member, set their auth role

SHOULD-tier (only if a gate passes early)

Method	Path	Description
PATCH	/tasks/{id}	Status, assignee or sprint placement
GET	/sprints	List sprints and which is active
POST	/admin/sprints	Create a sprint
GET	/admin/team	Team roster and roles
POST	/admin/team/{user_id}/role	Set a member's team role
POST	/agent/actions/{id}/undo	Revert using before_state

13. Guardrails (must-have)

- Every agent answer shows chunk-level citations; no answer without at least one citation unless explicitly marked general knowledge
- Groundedness self-check runs before any answer reaches the user; failing answers are flagged, never shown as confident fact
- Confidence is a stated, uncalibrated value with a defined formula (reranker score plus grading verdict), never an unexplained number
- Prompt injection defence is structural: document and tool-returned content is data, never instructions, and only the user's own turns can trigger a tool call. Proven with the 8-document red-team suite before Day 5, not asserted
- No agent-proposed write action reaches the workspace without explicit approval; there is no silent auto-execute path in Week 1
- Full audit trail: every prediction, every proposed and approved or rejected action, every admin decision, timestamped and attributable, agent actions included via actor_type

14. Risks & latency

- **Latency:** a five-stage pipeline, rewrite, hybrid retrieve, rerank, grade, retry, groundedness check, risks 10 to 20 seconds per question. Target p50 under 8 seconds; if missed, the retry loop is cut first, matching Gate G1
- **Demo risk:** a live 15 to 20 second wait on camera reads badly regardless of the number. Pre-warm the cache on the exact demo questions beforehand
- **Day 1 spikes needed before committing to the plan:** hybrid vector DB setup, PDF/Word parsing quality on real documents, Google OAuth consent flow

- **Schedule risk:** PRD sign-off and the golden-set write-up both sit on Day 1 alongside the spikes; if either slips, the golden set is the one that can move to Day 2 morning without cost, sign-off cannot

15. Why this is different

Notion Agent, ClickUp's Brain² and Tana all now complete multi-step actions inside a workspace, not just answer questions. “An AI that takes actions” stopped being a differentiator on its own some time in the last year.

The claim here is narrower and provable: depth of inspection. A source link is not the same as a chunk-level citation with character offsets. A confidence badge is not the same as a stated, defined formula. A guardrail claim is not the same as an 8-document red-team suite with a published pass rate.

That is the whole point of sections 4 and 5 existing: the differentiator is only real if it is measured, not asserted.

16. Five-day build plan

Day	Focus
1	Auth and workspace scaffolding. Spike: hybrid vector DB setup, document parsing quality check. Golden set of 15 questions hand-labelled. Scope (section 3) and gates (section 4) posted for sign-off.
2	Block-based note editor on mock data. Document upload and chunking pipeline live, writing to chunks. Red-team documents authored.
3	Full retrieval pipeline wired: hybrid search, rerank, grade, retry, groundedness check. Gate G1 checked against the golden set.
4	Agent loop live: propose → hold → approve/reject → write. Audit log wired. Gate G2 checked.
5	Admin review queue, pipeline health dashboard from retrieval_runs, red-team suite run and scored, bug bash, deploy, cache pre-warmed on demo questions, demo prep.

17. Open decisions

Gating — must close before Day 1

- None. Confidence formula, approval-not-auto-execute, and the latency cut rule are now decided in sections 4, 5 and 9.

Non-gating — can wait

- Final product name (**Meridian** is proposed)
- Which **SHOULD** item, backlog/sprint, auto-assignment or undo, gets picked up first if a gate passes with room to spare